

## Conceptual Architecture Description and Evaluation Report (T2)

Delivery Date: May 22, 2026

Team Members: Rukiye Sedef Keskin

Supervisor: Prof. Ayça Kolukısa

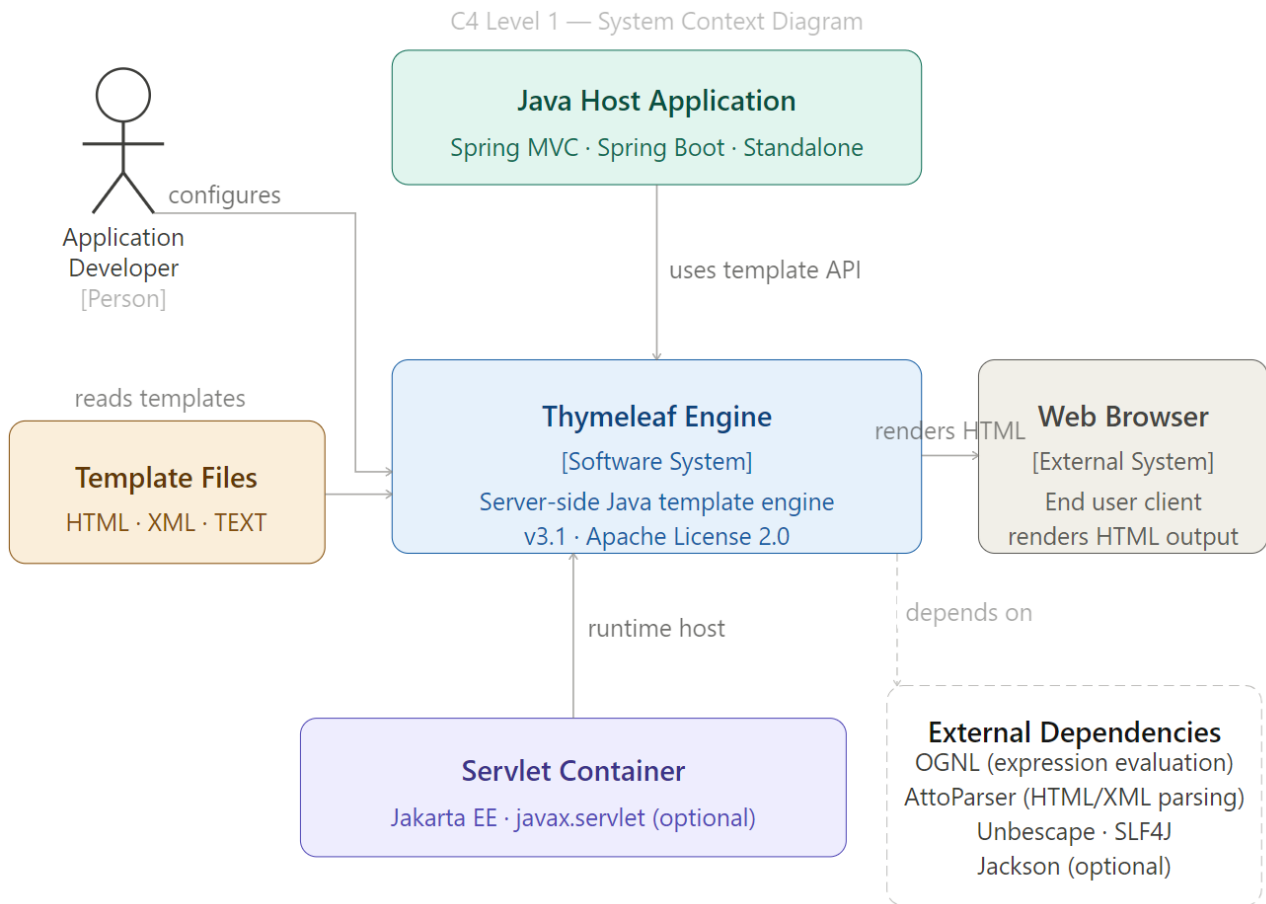
(<https://avesis.hacettepe.edu.tr/atarhan>)

### 1. OSS PRODUCT OVERVIEW

**Table 1.** Description of the OSS product

|                             |                                                                                                                                                                                               |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Name of OSS Product:        | <b>Thymeleaf</b>                                                                                                                                                                              |
| Description of OSS product: | Thymeleaf is a modern server-side Java template engine developed for both web and standalone environments. Its main purpose is to introduce 'natural templates' into the development process. |
| URL of OSS product:         | <a href="https://github.com/thymeleaf/thymeleaf">https://github.com/thymeleaf/thymeleaf</a>                                                                                                   |
| Size of OSS product (KLOC): | 110KLOC                                                                                                                                                                                       |

### 2. CONTEXT DIAGRAM



### 3. CONCRETE ARCHITECTURE EVALUATION SUMMARY

- **Coupling:** The coupling between classes in Thymeleaf is generally at an acceptable level. The median CBO is 7, which stays below the commonly accepted threshold of 10. The CF value is 21%, indicating that the majority of classes operate independently from each other.
- **Complexity:** The complexity per class in terms of methods (WMC = 5) is low and healthy. However, the average cyclomatic complexity per method (CC = 12.33) is slightly high and may make some methods harder to test. The maximum nesting depth (NNL = 8) may also negatively affect code readability
- **Cohesion:** With a median LCOM of 3, the methods within classes do not always fully share the same data. A median NOM of 4 shows that classes have not grown unnecessarily large. Overall, cohesion is neither particularly good nor particularly bad.
- **Inheritance:** Inheritance is used quite sparingly; only 18% of classes have any subclasses. This suggests that composition is preferred over inheritance. However, some classes with a maximum DIT of 6 and a maximum MIF of 123 have very deep inheritance chains, which may cause maintenance issues.
- **Size & Volume:** Classes are quite small and focused (median LOC = 69, median SLOC = 37). Approximately half of the code consists of comments and blank lines, which indicates that documentation is well maintained.
- **Encapsulation:** 72% of attributes are hidden from external access, which is a good result. However, only 16% of methods are defined as private or protected, meaning the vast majority are public. Since Thymeleaf is a framework, a wide public API is expected, but some internal methods appear to be unnecessarily public.
- **Overall Maintainability:** With a median MI of 73.27, Thymeleaf falls into the "good" maintainability category. If the CC and MHF values are improved, this score could be pushed even higher.
- **Dependency Analysis:** The dependency analysis revealed the most important finding: the **<engine>** package has circular dependencies with **the <util>**, **<model>**, **<cache>**, and **<context>** packages. These cycles make it harder to test and develop these packages independently. The **<util>** package depends on almost no other package, yet nearly every package in the system depends on it, making it a risky package to modify. The **<templatemode>** package carries a similar risk, as it receives dependencies from more than 20 packages.
- **Informal Observations / Difficulties:**
  - While interpreting the dependency matrix, I initially confused the row and column directions for the **<util>** and **<exceptions>** packages. Examining the dependency graphs allowed me to arrive at the correct interpretation.
  - In a project with 1,901 classes, analyzing everything based on median metric values made it harder to notice the few classes that were actually problematic.

### 4. TOOLS USED FOR CONCEPTUAL ARCHITECTURE DESCRIPTION

**Table 2.** The list of tools used for architectural description

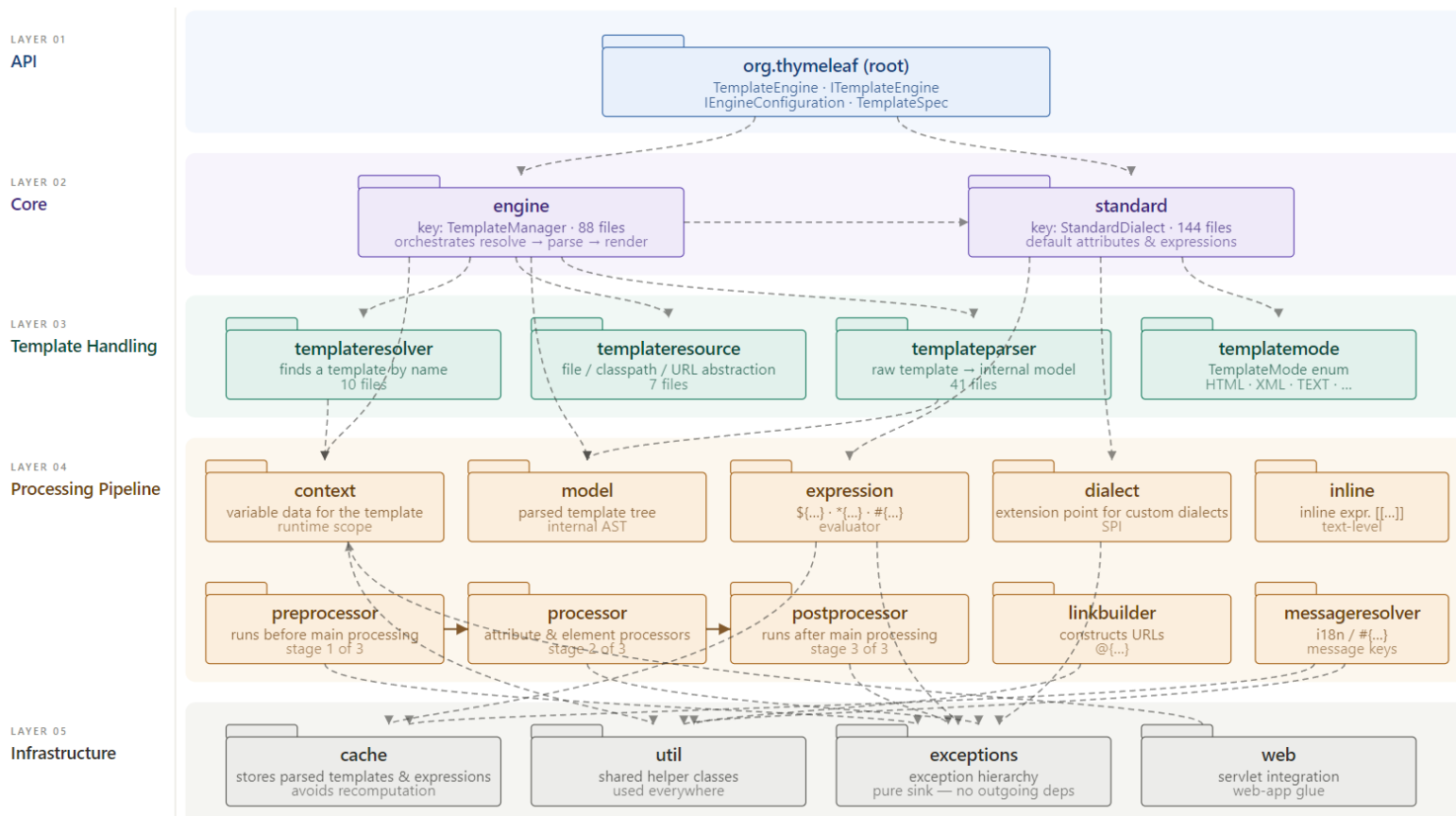
| Tool Name | Purpose of Use | URL |
|-----------|----------------|-----|
|-----------|----------------|-----|

|                                  |                                                                 |                                                                                                                                                                                                                                                                                                          |
|----------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SciTool Understand               | To obtain maintainability metrics, To create dependency graphs  | <a href="https://scitools.com/">https://scitools.com/</a>                                                                                                                                                                                                                                                |
| C&K Extended                     | To obtain maintainability metrics                               | <a href="https://gromit.iar.pwr.wroc.pl/p_inf/ckjm/">https://gromit.iar.pwr.wroc.pl/p_inf/ckjm/</a>                                                                                                                                                                                                      |
| GitHub                           | Accessing and reviewing Thymeleaf source code and documentation | <a href="https://github.com/thymeleaf/thymeleaf">https://github.com/thymeleaf/thymeleaf</a>                                                                                                                                                                                                              |
| Thymeleaf Official Documentation | Understanding conceptual components and their roles             | <a href="https://www.thymeleaf.org/documentation.html">https://www.thymeleaf.org/documentation.html</a>                                                                                                                                                                                                  |
| Sonarqube                        | Retreiving Code Smells and Descriptions                         | <a href="https://www.sonarsource.com/products/sonarqube/">https://www.sonarsource.com/products/sonarqube/</a>                                                                                                                                                                                            |
| Lucidchart & Claude Design       | Drawing context diagram and conceptual architecture diagrams    | <a href="https://www.lucidchart.com/pages/use-cases/creating-technical-diagrams">https://www.lucidchart.com/pages/use-cases/creating-technical-diagrams</a><br><br><a href="https://www.anthropic.com/news/claude-design-anthropic-labs">https://www.anthropic.com/news/claude-design-anthropic-labs</a> |

## 5. DESCRIPTION OF MODULE VIEW(S)

### 5.1. Module View-1: Thymeleaf UML Package Diagram(s)

The module view of Thymeleaf is represented as a UML package diagram organized into five architectural layers. Each layer groups packages that share a similar responsibility, and dependencies flow from upper layers down to lower layers via «use» relationships (shown as dashed arrows in the diagram).



- **Layer 1 — API (`org.thymeleaf` root)** This is the top-most layer and serves as the public entry point of the entire system. It contains the core public-facing classes: `<TemplateEngine>`, `<ITemplateEngine>`, `<EngineConfiguration>`, and `<TemplateSpec>`. External applications interact with Thymeleaf exclusively through this layer. It depends on the layers below it but is not depended upon by any other package, making it the cleanest boundary in the system.
- **Layer 2 — Core (`engine`, `standard`)** This layer contains the two largest and most central packages in the system. The `<engine>` package (88 files, key class: `<TemplateManager>`) is responsible for orchestrating the entire template processing lifecycle — it coordinates template resolution, parsing, and rendering. The `<standard>` package (144 files, key class: `<StandardDialect>`) provides Thymeleaf's default dialect, which defines the standard set of attributes and expressions available to template authors. Both packages are heavily used by the rest of the system, making this layer the architectural core.
- **Layer 3 — Template Handling (`templateparser`, `templatere resolver`, `templatere source`, `templatemode`)** This layer handles everything related to locating, reading, and categorizing templates before processing begins. `<templatere solver>` (10 files) decides how to find a template given its name. `<templatere source>` (7 files) abstracts the actual resource (e.g., file, classpath, URL). `<templateparser>` (41 files) parses the raw template content into an internal model. `<templatemode>` defines the `<TemplateMode>` enum (HTML, XML, TEXT, etc.) that is used system-wide to distinguish template types.
- **Layer 4 — Processing Pipeline (`context`, `model`, `expression`, `dialect`, `inline`, `processor`, `preprocessor`, `postprocessor`, `linkbuilder`, `messengeresolver`)** This is the largest layer and represents the active processing pipeline. During template execution, the `<context>` package holds the variable data available to the template. The `<model>` package represents the internal tree structure of a parsed template. The `<expression>` package evaluates Thymeleaf expressions (e.g., `{...}`, `*{...}`). The `<processor>` package contains attribute and element processors that apply transformations to the model. `<preprocessor>` and `<postprocessor>` handle steps that run before and after the main processing phase. `<inline>` handles inline expression processing (e.g., `[[...]]`). `<dialect>` defines the extension points for custom dialects. `<linkbuilder>` constructs URLs, and `<messengeresolver>` handles internationalization messages.
- **Layer 5 — Infrastructure (`cache`, `util`, `exceptions`, `web`)** This is the bottom-most layer and provides cross-cutting support to all other layers. `<cache>` stores parsed templates and expressions to avoid redundant computation. `<util>` contains shared utility classes used across the entire codebase. `<exceptions>` defines the system's exception hierarchy and is a pure dependency sink (no outgoing dependencies). `<web>` provides servlet-specific implementations for web-based use cases.
- **Architectural Pattern:** The layered organization of these packages follows a **Layered Architecture** pattern, where each layer has a well-defined responsibility and depends only on the layers below it. This promotes separation of concerns and makes the system easier to understand and maintain. Additionally, the design of the `<engine>` package as a central coordinator that delegates to specialized subsystems is consistent with a **Pipes and Filters** pattern, particularly visible in the sequential flow through `<preprocessor>` → `<processor>` → `<postprocessor>`.

## 5.2. Traceability of the Module View to the Dependency Graph(s)

The package diagram presented in Section 5.1 is directly traceable to the dependency graphs provided in Appendix C of the T1 report. The table below maps each architectural layer in the module view to the corresponding dependency graph(s).

**Table 3.** Traceability of the module view(s) to the dependency graph(s)

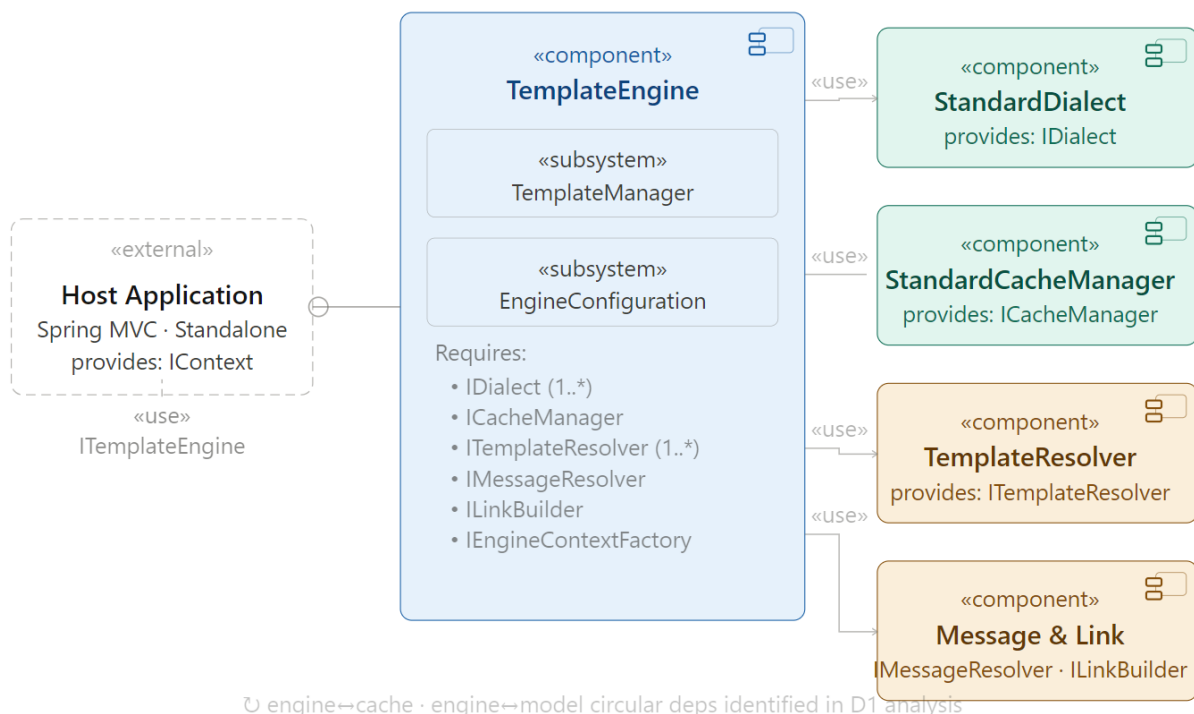
| Module View (Layer/Package)                           | Dependency Graph (from T1 Appendix C) |
|-------------------------------------------------------|---------------------------------------|
| Layer 2 – Core: <code>&lt;engine&gt;</code>           | Graph 1 (C4 L3), Graph 2 (Engine L4)  |
| Layer 5 – Infra: <code>&lt;exceptions&gt;</code>      | Graph 3 (Exceptions L4)               |
| Layer 4 – Pipeline: <code>&lt;inline&gt;</code>       | Graph 5 (Inline L4)                   |
| Layer 4 – Pipeline: <code>&lt;linkbuilder&gt;</code>  | Graph 6 (Linkbuilder L4)              |
| Layer 4 – Pipeline: <code>&lt;preprocessor&gt;</code> | Graph 7 (Preprocessor L4)             |

|                                                                  |                                |
|------------------------------------------------------------------|--------------------------------|
| Layer 4 – Pipeline: <b>&lt;postprocessor&gt;</b>                 | Graph 8 (Postprocessor L4)     |
| Layer 4 – Pipeline: <b>&lt;messageresolver&gt;</b>               | Graph 9 (Messageresolver L4)   |
| Layer 4 – Pipeline: <b>&lt;model&gt;</b>                         | Graph 10 (Model L4)            |
| Layer 4 – Pipeline: <b>&lt;processor&gt;</b>                     | Graph 11 (Processor L4)        |
| Layer 4 – Template: <b>&lt;templatemode&gt;</b>                  | Graph 12 (Templatemode L4)     |
| Layer 3 – Template: <b>&lt;templateparser&gt;</b>                | Graph 13 (Templateparser L4)   |
| Layer 3 – Template: <b>&lt;templateresolver&gt;</b>              | Graph 14 (Templateresolver L4) |
| Layer 3 – Template: <b>&lt;templateresource&gt;</b>              | Graph 15 (Templateresource L4) |
| Layer 5 – Infra: <b>&lt;cache&gt;</b>                            | Graph 16 (Cache L4)            |
| Layer 2 – Core: <b>&lt;standard&gt;</b> / <b>&lt;dialect&gt;</b> | Graph 17 (Dialect L4)          |
| Layer 4 – Pipeline: <b>&lt;context&gt;</b>                       | Graph 18 (Context L4)          |
| Layer 5 – Infra: <b>&lt;web&gt;</b>                              | Graph 19 (Web L4)              |
| Layer 5 – Infra: <b>&lt;util&gt;</b>                             | Graph 20 (Util L4)             |
| All layers (system-wide overview)                                | Graph 1 (C4 L3)                |

The L3 graph (Graph 1) provides the best overall correspondence to the module view, as it shows all packages and their inter-package dependencies at the same level of abstraction as the package diagram. The individual L4 graphs provide a deeper trace for each specific package, confirming the dependency directions implied by the layered structure in the module view.

## 6. DESCRIPTION OF COMPONENT AND CONNECTOR (C&C) VIEW(S)

The Component & Connector (C&C) view presents the runtime components of Thymeleaf and the interaction mechanisms (connectors) between them. In contrast to the static package structure of the module view, the C&C view reveals *how the system operates* — which component provides which interface, and which component depends on which interface.



### Component Diagram:

- **TemplateEngine** TemplateEngine is the single, central entry point of Thymeleaf. It receives all incoming template processing requests (process() calls) from the host application and coordinates the template resolution, parsing, and rendering steps through its internal TemplateManager. It exposes the ITemplateEngine interface to external clients; the host application interacts with

Thymeleaf exclusively through this interface. All sub-component configurations are held via `IEngineConfiguration`. The circular dependencies identified in the T1 analysis — engine ↔ model (485/7), engine ↔ cache (16/42), engine ↔ context (34/108), and engine ↔ util (186/35) — remain the most critical architectural risk associated with this component.

- **StandardDialect** `StandardDialect` implements the `IDialect` interface and provides Thymeleaf's default set of processors and expressions (e.g., `th:text`, `th:each`, `${...}`). During initialisation, `TemplateEngine` requires one or more `IDialect` implementations; `StandardDialect` fulfils this requirement. Custom dialects can be integrated via the same `IDialect` interface, making this an extensibility point for the system.
- **StandardCacheManager** `StandardCacheManager` implements the `ICacheManager` interface and is responsible for managing both the template cache and the expression cache. By retaining previously parsed templates in memory, it eliminates redundant parsing overhead. The circular dependency between engine and cache (16/42) identified in the T1 dependency analysis indicates that this component maintains a bidirectional runtime relationship with `TemplateEngine`.
- **TemplateResolver** `TemplateResolver` implements the `ITemplateResolver` interface and abstracts access to the actual template resource — whether a file on disk, a classpath entry, or a remote URL — given a template name. When a processing request arrives, `TemplateEngine` first delegates to this component to locate the template. Multiple `TemplateResolver` instances may be configured in a priority chain, allowing flexible resource strategies.
- **Message & Link** This component represents two closely related, low-coupling packages under a single runtime boundary. `MessageResolver` (`IMessageResolver`) resolves `#{...}` expressions in templates by looking up internationalisation (i18n) message bundles. `LinkBuilder` (`ILinkBuilder`) transforms `@{...}` expressions into context-relative URLs. Both are required by `TemplateEngine` at configuration time and have no outgoing dependencies of their own, confirming their role as stable leaf components — as also evidenced by the T1 dependency graphs (Graphs 6 and 9).
- **Host Application (external)** The Host Application is an external actor outside of the Thymeleaf library boundary. It may be a Spring MVC View Resolver, a standalone Java process, or any other framework that embeds Thymeleaf. It is responsible for constructing an `IContext` implementation (carrying the template variable data) and initiating the template processing lifecycle by calling `TemplateEngine.process()`. It connects to Thymeleaf solely through the `ITemplateEngine` interface, which keeps the coupling between the host and the library clean and well-defined.

All connectors in this view are of the «use» type, realised through a **procedure call** mechanism — that is, synchronous method invocations across component boundaries.

**Table 4.** Traceability of the components in the C&C view(s) to the elements in the module view(s).

| Component(s) in C&C view(s) | Element(s) in module view(s)                                                                                                                                                                                                                                    |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TemplateEngine              | Layer 1 – API: <b>&lt;org.thymeleaf&gt;</b> root ( <code>TemplateEngine</code> , <code>ITemplateEngine</code> , <code>IEngineConfiguration</code> , <code>TemplateSpec</code> ); Layer 2 – Core: <b>&lt;engine&gt;</b> package ( <code>TemplateManager</code> ) |
| StandardDialect             | Layer 2 – Core: <b>&lt;standard&gt;</b> package ( <code>StandardDialect</code> ); Layer 4 – Pipeline: <b>&lt;dialect&gt;</b> package                                                                                                                            |
| StandardCacheManager        | Layer 5 – Infrastructure: <b>&lt;cache&gt;</b> package                                                                                                                                                                                                          |

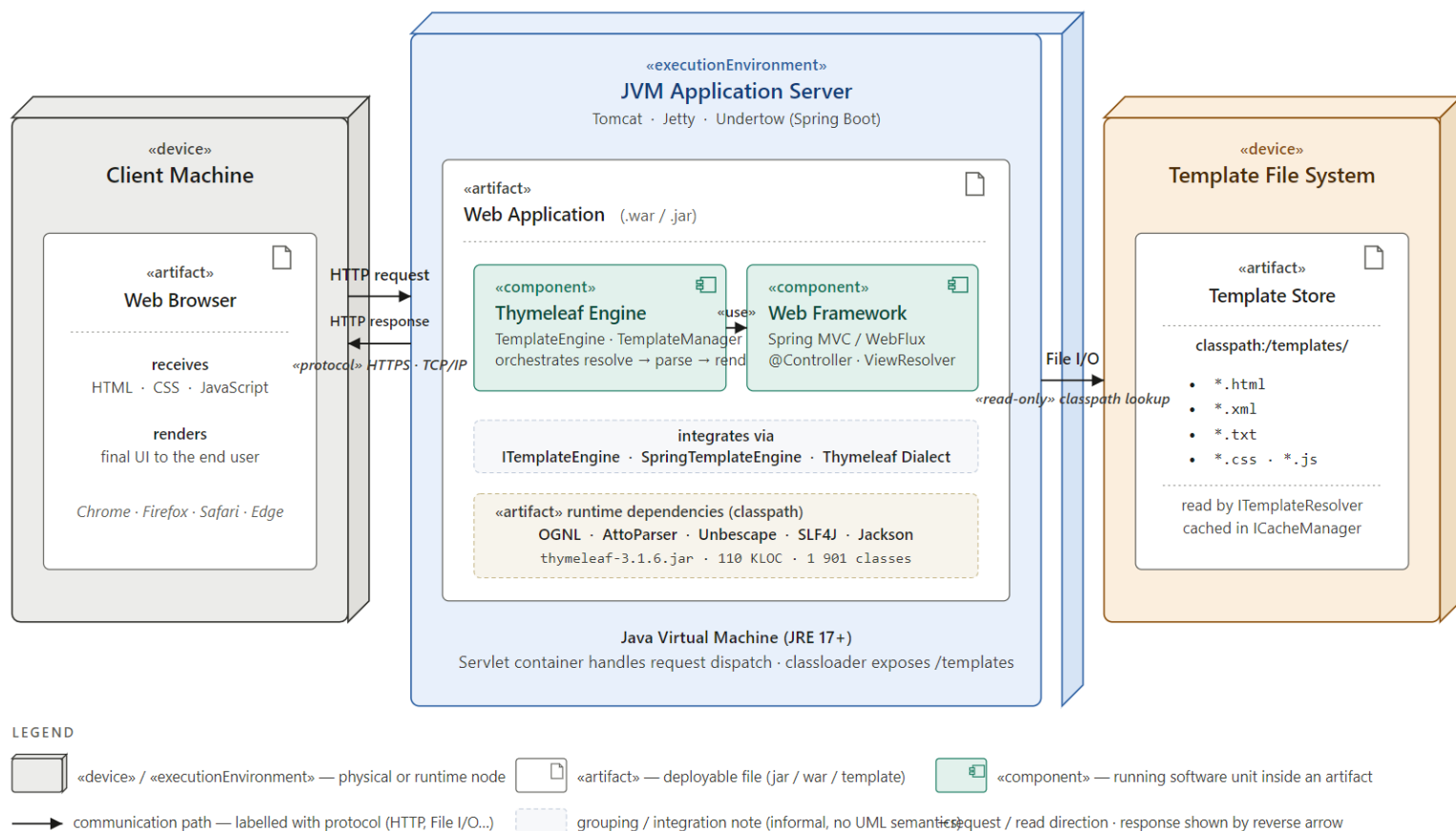
|                  |                                                                                                                                            |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| TemplateResolver | Layer 3 – Template Handling: <b>&lt;templateresolver&gt;</b> package; Layer 3 – Template Handling: <b>&lt;templateresource&gt;</b> package |
| Message & Link   | Layer 4 – Pipeline: <b>&lt;messageresolver&gt;</b> package; Layer 4 – Pipeline: <b>&lt;linkbuilder&gt;</b> package                         |
| Host Application | N/A (external actor; not part of Thymeleaf's own module view)                                                                              |

Table 4 maps each component in the C&C view to the corresponding packages in the module view. The general principle followed is that some packages which appear separately at module-view granularity are grouped under a single component in the C&C view, because they fulfil the same architectural role and exhibit negligible mutual coupling. The TemplateEngine component, on the other hand, spans two distinct layers in the module view — Layer 1 (the externally exposed API) and Layer 2 (the runtime implementation of that API) — reflecting its dual role as both the public contract and the central orchestrator of the system.

## 7. DESCRIPTION OF ALLOCATION VIEW (DEPLOYMENT DIAGRAM)

The allocation view maps the logical design elements identified in the module and C&C views onto the physical execution environment. It answers where packages and components execute and on which infrastructure they reside.

The deployment diagram of the Thymeleaf project is provided in the figure below. The diagram identifies three environmental nodes and the artifacts deployed onto them.



### Node Descriptions



- **Node 1 – JVM Application Server** («executionEnvironment»):

This is the primary execution environment of the Thymeleaf system. It runs the Web Application artifact — packaged as a **.war** or **.jar** — which hosts two components: the **Thymeleaf Engine** and the **Web Framework** (e.g., Spring MVC). These two components are integrated through the **ITemplateEngine** interface and the **@Controller** abstraction — the Web Framework routes an incoming HTTP request to a controller, which then initiates the **TemplateEngine.process()** call via **ITemplateEngine** to render the response.

The Thymeleaf Engine is deployed as **thymeleaf-3.1.6.jar** (110 KLOC, 1,901 classes) and brings the following runtime dependencies onto the classpath: **OGNL** (expression evaluation), **AttoParser** (HTML/XML parsing), **Unescape** (character escaping), **SLF4J** (logging), and **Jackson** (optional serialization support).

All five C&C components identified in Section 6 — **TemplateEngine**, **StandardDialect**, **StandardCacheManager**, **TemplateResolver**, and **Message & Link** — execute on this node. All five architectural layers of the module view (Layer 1–5) are likewise deployed here.

- **Node 2 – File System** («device»):

This node represents the persistent storage environment from which template files are read at request time. It contains the **Template Store** artifact, which holds the raw template files organised under the application classpath (**.html**, **.xml**, **.txt**). The connection from the JVM Application Server to this node is a **read-only File I/O** channel — Thymeleaf never writes back to template files at runtime. Once a template has been read and parsed, its compiled representation is stored in the **ICacheManager** (**StandardCacheManager**) on the JVM side, avoiding redundant file reads on subsequent requests.

This node corresponds to the **<templateresource>** package in Layer 3 of the module view. That package is responsible for abstracting the physical location of template files behind the **ITemplateResource** interface.

- **Node 3 – Client Machine** («device»):

This node represents the end-user's browser environment. It hosts the **Web Browser** artifact, which receives the fully rendered HTML, CSS, and JavaScript output produced by the Thymeleaf Engine and displays it to the user. The connection between the JVM Application Server and the Client Machine is an **HTTP/HTTPS** channel. Thymeleaf has no runtime presence on this node — all processing takes place entirely on the server side, and the output delivered to the browser is plain, static HTML that carries no Thymeleaf-specific syntax or attributes.

The allocation view confirms that Thymeleaf follows a **server-side rendering** model: the entire processing pipeline — from template resolution through expression evaluation to HTML generation — executes within a single JVM environment. The File System node is a passive dependency accessed exclusively by the **templateresource** package, and the Client Machine node is a pure output consumer with no coupling back into any Thymeleaf component.

## 8. DESCRIPTION OF SPECIFIC QUALITY ATTRIBUTE (QA) SCENARIO

*A developer working on the Thymeleaf project wants to add a new feature to the **<engine>** package. This is a straightforward update made during a normal development cycle. However, as identified in the T1 report, the **<engine>** package has mutual dependency relationships with the **<model>**, **<util>**, **<cache>**, and **<context>** packages. As a result, the developer cannot simply modify the **<engine>** package alone — they are also forced to inspect and update the other four packages accordingly. For this change to be considered successful, it is expected that no more than 2 packages are affected, the work is completed in under 4 hours, and all existing tests continue to pass. Under the current architecture, however, these criteria cannot be met.*

- **Source of stimulus:** A software developer working on Thymeleaf
- **Stimulus:** A request to add a new feature to the **<engine>** package



- **Environment:** Normal development process; a routine update task
- **Artifact:** The **<engine>** package and the **<model>**, **<util>**, **<cache>**, **<context>** packages that have dependency relationships with it
- **Response:** The developer makes the change, then identifies and updates the affected packages, and re-runs the existing tests
- **Response measure:** No more than 2 packages should be affected; the change should be completed in under 4 hours; no existing tests should break

**Table 5.** Traceability of the specific quality attribute scenario(s) to the question(s) and answer(s) in your QQM tree

| Specific QA scenario                                      | Question in QQM tree                                    | Explanation (or rationale)                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------|---------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The change should affect at most 2 packages               | <b>Q1</b> – What is the coupling level between classes? | Although general metrics such as <b>CBO = 7</b> suggest a reasonable level of coupling, the dependency graphs reveal that the <b>&lt;engine&gt;</b> package has mutual dependencies with <b>&lt;model&gt;</b> , <b>&lt;util&gt;</b> , <b>&lt;cache&gt;</b> , and <b>&lt;context&gt;</b> . This causes a single change to propagate in a chain-like manner, directly challenging the "at most 2 packages" response measure. |
| The change should be completed in under 4 hours           | <b>Q2</b> – What is the complexity level of the code?   | <b>CC = 12.33</b> and <b>NNL = 8</b> indicate that understanding the code is not straightforward. Halstead metrics ( <b>H_D = 31.88</b> , <b>H_E = 74.25</b> ) further suggest that a developer needs to invest considerable time to fully comprehend the codebase, making the 4-hour time constraint difficult to meet.                                                                                                   |
| All existing tests should continue to pass                | <b>Q3</b> – Is class cohesion sufficient?               | <b>LCOM = 3</b> shows that the methods within a class do not always operate on the same variables, which makes writing tests more difficult and maintaining existing test coverage harder. <b>RFC = 13</b> also indicates that the number of scenarios to be tested per class is not negligible.                                                                                                                           |
| The scenario is also aligned with overall maintainability | <b>Q7</b> – What is the overall maintainability level?  | <b>MI = 73.27</b> places Thymeleaf in the "good" band, yet still below the "excellent" threshold ( $MI \geq 85$ ). This scenario provides a concrete and measurable context in which the improvements suggested in the T1 report — particularly regarding complexity and encapsulation — can be put into practice.                                                                                                         |

## 9. SUGGESTIONS FOR IMPROVEMENT OF THE PRODUCT ARCHITECTURE

### 9.1 Suggestions Based on Static Analysis Findings

The most critical finding of the T1 concrete architecture analysis is that the **<engine>** package sits at the very centre of the system and is involved in bidirectional circular dependencies with the **<model>**, **<util>**, **<cache>**, and **<context>** packages:

- **<engine>** ↔ **<model>** (485 / 7)
- **<engine>** ↔ **<util>** (186 / 35)
- **<engine>** ↔ **<cache>** (16 / 42)
- **<engine>** ↔ **<context>** (34 / 108)

These cycles mean that none of these packages can be understood, tested, or modified in isolation. Any change made to the **<engine>** package propagates to the other four packages, and vice versa. This is

precisely the condition described in the QA scenario in Section 8: a developer adding a new feature to the **<engine>** package is forced to inspect and update four additional packages simultaneously, making it impossible to satisfy the response measure of "no more than 2 packages affected."

A second factor that compounds the difficulty is code-level complexity. The T1 analysis revealed a median cyclomatic complexity of  $CC = 12.33$  and a maximum nesting depth of  $NNL = 8$ . This means that while the developer is already forced to touch multiple packages due to circular dependencies, they also encounter heavily branching and deeply nested methods within each of those packages. The Halstead metrics ( $H_D = 31.88$ ,  $H_E = 74.25$ ) confirm that understanding these methods requires considerable mental effort. Together, the two problems make the 4-hour response measure in the QA scenario unachievable under the current architecture.

The two suggestions below are both grounded in this shared rationale — one addressing the structural dimension of the problem, and the other addressing the code quality dimension.

**References:** T1 Dependency Graphs (Graphs 1, 2, 10, 16, 18, 20), T1 GQM Q2 ( $CC = 12.33$ ,  $NNL = 8$ ,  $H_D = 31.88$ ), C&C View — TemplateEngine component (Section 6), Module View — Layers 2, 4, and 5 (Section 5.1), QA Scenario (Section 8).

- **Suggestion-1: Breaking the Circular Dependencies:**

A narrow interface layer should be introduced between **<engine>** and the packages with which it shares circular dependencies:

- An `IEngineContext` interface should be defined and implemented by **<context>**, so that **<engine>** depends on the abstraction rather than the concrete class.
- Similarly, `ITemplateModel` and `ICacheAccess` interfaces should be introduced so that **<engine>** depends on abstractions rather than the concrete **<model>** and **<cache>** packages.
- The 35-reference back-dependency from **<util>** to **<engine>** should be reduced by moving the relevant utility calls into a dedicated internal helper class owned by **<engine>** itself.

This approach follows the **Dependency Inversion Principle** and is consistent with the interface-based design already present in the C&C view (`ITemplateEngine`, `ICacheManager`, `ITemplateResolver`), extending the same philosophy inward to the system's internal layers.

- **Suggestion-2: Reducing Cognitive Complexity:**

The **Extract Method** refactoring pattern should be applied to any method with  $CC > 15$  or  $NNL > 4$ . Each deeply nested block should be extracted into a smaller, clearly named private method that communicates its intent. Additionally, deep nested conditional chains should be replaced with **guard clauses** (early returns) wherever applicable, as this reduces nesting depth without changing behaviour.

These refactorings do not alter the external interface of any component and therefore carry no risk of breaking existing behaviour. They directly improve testability and reduce the time a developer needs to safely understand and modify the code — making the 4-hour response measure in the QA scenario achievable, especially when applied together with Suggestion-1.

## 9.2 Suggestions Based on Code Smell Analysis

SonarQube classifies detected code smells into five severity levels. The definitions of these levels and their distribution in the Thymeleaf analysis are presented in the table below.

**Table: SonarQube Severity Levels and Distribution in Thymeleaf**

| Severity | Definition                                                                                                                                                               | Count in Thymeleaf |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| BLOCKER  | Findings that may directly affect application behavior and carry a high risk of causing defects or security vulnerabilities in production. Immediate action is required. | 8                  |
| CRITICAL | Findings that do not break application logic but significantly degrade code quality and increase the risk of future defects. Should be addressed with high priority.     | 306                |
| MAJOR    | Findings that negatively affect maintainability, testability, and readability without causing immediate behavioral failures. Should be addressed with medium priority.   | 1,224              |
| MINOR    | Small improvement opportunities that do not alter functionality but indicate non-compliance with modern Java practices and style conventions.                            | 1,421              |
| INFO     | Informational markers with no direct risk or defect; observations that are recommended to be aware of.                                                                   | 860                |
| TOTAL    |                                                                                                                                                                          | 3,819              |

Examining the distribution of the 3,819 findings, MINOR and MAJOR categories are dominant. The fact that the BLOCKER count is as low as 8 is a positive indicator of Thymeleaf's overall code quality; however, each of these 8 findings represents a concrete issue that requires individual attention.

### 9.2.1 General Improvement Suggestions for Non-Blocker Code Smells

The following suggestions are presented by grouping CRITICAL, MAJOR, MINOR, and INFO findings thematically. Each group is also related to the static metric findings from the T1 report.

#### **Group 1 — Cognitive Complexity (java:S3776, CRITICAL, 165 occurrences)**

This is the most frequently flagged CRITICAL finding type in the system. SonarQube's Cognitive Complexity metric measures how difficult a method is to understand; the permitted threshold is 15. The analysis reveals that 165 methods exceed this threshold. The most extreme cases are:

| Class                                             | Cognitive Complexity    |
|---------------------------------------------------|-------------------------|
| AbstractStandardFragmentInsertionTagProcessor     | 68 (4.5x the threshold) |
| AbstractMarkupTemplateParser                      | 41                      |
| SpringSelectedValueComparator (Spring5 & Spring6) | 34                      |
| ExpressionUtils                                   | 17                      |
| FragmentExpression                                | 16                      |

This finding directly aligns with the T1 report results of CC = 12.33 (median) and NNL = 8 (maximum). Additionally, 22 "Brain Method" findings at INFO level (java:S6541) reinforce this concern — these methods simultaneously exhibit high LOC, high complexity, deep nesting, and a large number of variables.

**Suggestion:** The Extract Method refactoring pattern should be applied to all methods whose cognitive complexity exceeds 15. Guard clauses (early returns) should be used in deeply nested if-else chains to reduce nesting depth. Methods whose complexity exceeds 30 should be decomposed into multiple smaller, focused methods.

## **Group 2 — Unnecessary 'public' Access Modifiers (java:S5786, INFO, 807 occurrences)**

This is the highest-volume finding type in the system. 807 methods are flagged as public while residing in a context that does not require this level of access. This finding directly supports and numerically explains the most significant encapsulation finding in the T1 report — MHF = 0.16, meaning only 16% of methods are private or protected.

**Suggestion:** Since Thymeleaf is a framework, having a wide public API surface is architecturally defensible; however, the methods flagged by these 807 findings fall outside that justification. The access modifiers of these methods should be systematically reviewed, and internal utility methods that are not part of the public API should be restricted to protected or private. This step directly aligns with the MHF improvement proposed in Section 9.1.

## **Group 3 — Null Return Anti-Pattern (java:S1168, MAJOR, 248 occurrences)**

248 methods return null where they should return an empty collection. This forces every caller of these methods to perform a null check; if the check is overlooked, a NullPointerException risk arises. This finding also exerts indirect pressure on CBO and RFC values, since defensive null checks increase inter-class coupling.

**Suggestion:** The return value of all collection-returning methods that currently return null should be replaced with Collections.emptyList(), Collections.emptyMap(), or Collections.emptySet(). This change does not alter behavior but eliminates unnecessary null checks on the calling side, improving readability.

## **Group 4 — Unused Code (java:S1068 MAJOR 139 + java:S1172 MAJOR 41 + java:S1481 MINOR 12 + java:S1144 MAJOR 6)**

Four different rules point to dead code within the system: 139 unused private fields (S1068), 41 unused method parameters (S1172), 12 unused local variables (S1481), and 6 unused private methods (S1144). Dead code makes it harder for developers to understand what is actually active, may negatively affect the LCOM value, and pushes up the H\_D = 31.88 (Halstead Difficulty) value identified in T1.

**Suggestion:** All unused fields, parameters, and methods should be removed. It is recommended that this cleanup is performed on a tested branch first and then validated by re-running the existing test suite.

## **Group 5 — String Literal Duplication (java:S1192, CRITICAL, 92 occurrences)**

In 92 locations, string literals have been written directly into the code rather than being defined as constants. For example, the string "Message cannot be null or empty" is repeated 5 times within a single class. This means that whenever such a string needs to change, all repetitions must be found and updated individually, increasing the risk of inconsistent modifications.

**Suggestion:** All repeated string literals should be defined as private static final String constants, and all usage points should be redirected to these constants.

## **Group 6 — Serialization Risk (java:S1948, CRITICAL, 20 occurrences)**

In 20 classes, non-static fields of classes that implement the Serializable interface are neither marked as transient nor implement Serializable themselves. This can cause a NotSerializableException when these objects need to be serialized.

**Suggestion:** The affected fields should either be marked with the transient keyword, or their types should be refactored to implement the Serializable interface.

### **Group 7 — Unexplained Empty Method Bodies (java:S1186, CRITICAL, 19 occurrences)**

19 methods have completely empty bodies with no accompanying explanation. An empty method body creates ambiguity about whether the method was intentionally left empty or whether its implementation was simply forgotten.

**Suggestion:** Methods that are intentionally left empty should have a `// intentionally empty` comment added to them. For implementations that are expected but not yet complete, throw new `UnsupportedOperationException("Not yet implemented")` should be used.

### **Group 8 — Deprecated API Usage (java:S5738, MAJOR, 86 occurrences)**

86 call sites reference classes and methods that have been marked for removal (`forRemoval = true`). As Java versions advance, these calls risk causing compilation or runtime errors.

**Suggestion:** The affected call sites should be migrated to the replacement API recommended in the relevant class's Javadoc. If migration is not immediately possible, a `@SuppressWarnings` annotation documenting the reason for continued use should be added.

### **Group 9 — Modern Java Practices (various MINOR rules, ~939 occurrences combined)**

Multiple MINOR rules indicate that several modern Java language features have not yet been adopted across the codebase:

| Rule       | Count | Description                                                                                                      |
|------------|-------|------------------------------------------------------------------------------------------------------------------|
| java:S2293 | 466   | <code>new ArrayList&lt;String&gt;()</code> should use the diamond operator: <code>new ArrayList&lt;&gt;()</code> |
| java:S1130 | 144   | throws Exception declarations that can never actually be thrown                                                  |
| java:S7158 | 128   | <code>sb.length() == 0</code> should use <code>sb.isEmpty()</code>                                               |
| java:S3878 | 113   | Ambiguous method calls requiring a cast to <code>Object</code> or <code>Object[]</code>                          |
| java:S4201 | 44    | Redundant null check before <code>instanceof</code> (which already returns false for null)                       |

**Suggestion:** These updates can largely be applied automatically using IDE refactoring tools (IntelliJ IDEA's "Inspect and Transform" or SonarLint's auto-fix feature). The changes carry no behavioral risk.

### **Group 10 — Test Quality (java:S5961, S5785, S5976, MAJOR, ~151 occurrences combined)**

Three MAJOR rules specific to test files have been identified: 64 methods contain more than 25 assertions in a single test method (S5961); 48 `assertEquals` call sites have their (expected, actual) argument order reversed (S5785); 39 repetitive test methods could be consolidated using `@ParameterizedTest` (S5976).

**Suggestion:** Methods containing excessive assertions should be split into smaller, focused test methods; the (expected, actual) parameter order in `assertEquals` calls should be corrected; and repetitive tests should be rewritten using `@ParameterizedTest`.

## **9.2.2 Detailed Fix Suggestions for Blocker Code Smells**

The 8 BLOCKER findings identified by SonarQube are capable of directly causing behavioral failures or security vulnerabilities in a production environment. Each is explained in detail below.

**Blocker-1 — Empty Test Class (java:S2187)**

|                |                                           |
|----------------|-------------------------------------------|
| <b>File</b>    | tests/.../engine/CloseElementTagTest.java |
| <b>Line</b>    | 31                                        |
| <b>Message</b> | Add some tests to this class.             |

Problem: The CloseElementTagTest class is defined as a JUnit test class but contains no test methods. The test runner may report this class as passing successfully — meaning a component appears to have been tested when it has not been tested at all.

**Fix:** At least one valid `@Test` method covering the functionality of the CloseElementTag component should be added. Alternatively, if the class is intentionally left empty, it should either be deleted or annotated with `@Disabled` to indicate that its implementation is deferred.

**Blocker-2 — Bitwise & Operator Used as Logical Operator (java:S2178)**

|                |                                                   |
|----------------|---------------------------------------------------|
| <b>File</b>    | tests/.../templateparser/text/TextTraceEvent.java |
| <b>Line</b>    | 103                                               |
| <b>Message</b> | Correct this "&" to "&&".                         |

Problem: The `&` operator is used in place of `&&` in a boolean expression. The `&&` operator performs short-circuit evaluation — if the left-hand side is false, the right-hand side is never evaluated. The `&` operator always evaluates both sides; if the right-hand side contains a null reference when the left-hand side is false, a `NullPointerException` may be thrown.

**Fix:** The `&` operator on the relevant line should be replaced with `&&`. This is a 5-minute fix that prevents a potential runtime failure.

**Blocker-3 — Field Shadowing (java:S2387) — 2 Occurrences**

|                |                                                      |
|----------------|------------------------------------------------------|
| <b>File 1</b>  | lib/thymeleaf-spring6/.../ThymeleafReactiveView.java |
| <b>File 2</b>  | lib/thymeleaf-spring5/.../ThymeleafReactiveView.java |
| <b>Line</b>    | 105 (in both)                                        |
| <b>Message</b> | "logger" is the name of a field in "AbstractView".   |

Problem: The ThymeleafReactiveView subclass declares a field named `logger` that already exists in its parent class `AbstractView`. This causes field shadowing: subclass methods use the subclass's own `logger` while super-scope methods continue using the parent's `logger`. This inconsistency makes it ambiguous which class a log record originates from. The same defect appearing in both Spring 5 and Spring 6 modules indicates propagation through copy-paste.

**Fix:** The `logger` field declared in the subclass should be removed, and the inherited `logger` field from the parent class should be used directly. If the parent's `logger` is not accessible (declared private), its access modifier should be updated to protected.

**Blocker-4 — Defective clone() Implementation (java:S2975) — 3 Occurrences**

|               |                                                               |
|---------------|---------------------------------------------------------------|
| <b>File 1</b> | lib/thymeleaf/.../StandardJavaScriptSerializer.java, line 489 |
| <b>File 2</b> | lib/thymeleaf/.../OGNLExpressionObjectsWrapper.java, line 138 |
| <b>File 3</b> | lib/thymeleaf/.../CharArrayWrapperSequence.java, line 110     |

|                |                                                                                     |
|----------------|-------------------------------------------------------------------------------------|
| <b>Message</b> | Remove this clone() implementation; use a copy constructor or copy factory instead. |
|----------------|-------------------------------------------------------------------------------------|

Problem: Three classes provide a copy mechanism using Java's Cloneable interface and Object.clone(). This mechanism is brittle: the clone() contract is inconsistently defined in Java; mutable fields are shallow-copied leading to shared references and unexpected side effects; and if super.clone() is not invoked, a consistent copy cannot be constructed. Notably, CharArrayWrapperSequence from the util package appears in this list — a package consumed by virtually the entire system, as confirmed by T1's dependency analysis.

**Fix:** For all three classes: the clone() method should be removed and replaced with a copy constructor or a static factory method (of(source)) that provides equivalent deep-copy behavior. The implements Cloneable declaration should also be removed from all three classes.

### Blocker-5 — Field Name Clash: empty / EMPTY (java:S1845)

|                |                                                                                |
|----------------|--------------------------------------------------------------------------------|
| <b>File</b>    | lib/thymeleaf/.../engine/IteratedGatheringModelProcessable.java                |
| <b>Line</b>    | 715                                                                            |
| <b>Message</b> | Rename field "empty" to prevent any misunderstanding/clash with field "EMPTY". |

Problem: The class declares both a static constant named EMPTY and an instance variable named empty. While Java is case-sensitive and the compiler distinguishes them, they are extremely confusing to developers. Mistakenly mixing up the two can lead to silent logic errors. Given that this class resides within the engine package — the package with the highest dependency values in the T1 report (485/7) — the blast radius of any error here is considerable.

**Fix:** The empty instance variable should be renamed to something that clearly communicates its intent, such as isCurrentIterationEmpty or currentEntryIsEmpty. This rename must be propagated consistently across all methods within the class.

## 10. REFERENCES

1. Thymeleaf Project. *Thymeleaf Source Code Repository*. GitHub. <https://github.com/thymeleaf/thymeleaf>
2. Thymeleaf Project. *Thymeleaf Official Documentation*. <https://www.thymeleaf.org/documentation.html>
3. SciTools. *Understand — Static Analysis and Code Metrics Tool*. <https://scitools.com/>
4. Spinellis, D. *CKJM Extended — Chidamber and Kemerer Java Metrics*. [https://gromit.iar.pwr.wroc.pl/p\\_inf/ckjm/](https://gromit.iar.pwr.wroc.pl/p_inf/ckjm/)
5. SonarSource. *SonarQube — Static Code Analysis Platform*. <https://www.sonarsource.com/products/sonarqube/>
6. SonarSource. *SonarQube Java Rules Documentation*. <https://rules.sonarsource.com/java/>
7. Chidamber, S. R., & Kemerer, C. F. (1994). A metrics suite for object oriented design. *IEEE Transactions on Software Engineering*, 20(6), 476–493.



8. Halstead, M. H. (1977). *Elements of Software Science*. Elsevier Science Inc.
9. Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley Professional.
10. Brown, S. *The C4 Model for Visualising Software Architecture*. <https://c4model.com/>
11. Martin, R. C. (2002). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall. (*Dependency Inversion Principle*)
12. Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley Professional. (*Extract Method, Guard Clauses*)
13. Keskin, R. S. (2026). **BBM485 Concrete Architecture Evaluation Report (T1) — Thymeleaf**. Hacettepe University, Department of Computer Engineering.

## 11. ALLOCATION OF RESPONSIBILITIES WITHIN TEAM MEMBERS

| Name of Team Member | Description of Responsibility | Allocation Unit |
|---------------------|-------------------------------|-----------------|
| Rukiye Sedef Keskin | All                           | All             |

12. APPENDIX-A: Thymeleaf Context Diagram and UML Package Diagram
13. APPENDIX-B: Thymeleaf Component Diagram and Deployment Diagram
14. APPENDIX-C: Dependency Graph(s) from OSS Concrete Architecture (T1 Report)